

Speed Controlled Cobot with E-Stop

Presented by
Matt Kwan

Presented for
Analog Garage
August 2025

Introduction & Approach

- Objective:
 - Design and implement a modular control system that adjusts the cobot's speed based on proximity input and responds immediately to an emergency stop signal.
- Tech stack:
 - C++, Python, ROS2
 - Forked from UR5 Simulation
- Components:
 - Proximity Sensor Node (Python)
 - E-Stop Controller Node (Python)
 - Speed Control Node (C++)
 - Publish Trajectory Node (C++, from UR5)
- Key features:
 - 3 speed states: FULL_SPEED, SLOW, STOP
 - E-Stop toggle with saved state
 - Automatic proximity detection with simulation object
 - Proximity hysteresis via window averaging
- Next: Results & Demonstration

```
ubuntu@e6fb7a587007: /home/mkwan/speed_controlled_ros2_ur5_interface
ubuntu@e6fb7a587007: /home/mkwan/speed_controlled_ros2_ur5_interface
ubuntu@e6fb7a587007: /home/mkwan/speed_controlled_ros2_ur5_interface
[500335 -0.320907 -0.011991]
[speed_control_node-18] [INFO] [1755489159.073511097] [speed_control_node]: Trajectory point 10: -1.473288 -1.213150 -1.933234 -
1.576946 -0.534089 -0.009993
[speed_control_node-18] [INFO] [1755489159.073517367] [speed_control_node]: Trajectory point 11: -1.460630 -1.162520 -1.906587 -
1.653557 -0.747272 -0.007994
[speed_control_node-18] [INFO] [1755489159.073523417] [speed_control_node]: Trajectory point 12: -1.447973 -1.111890 -1.879940 -
1.730167 -0.960454 -0.005996
[publish_trajectory_node-17] [INFO] [1755489159.073500317] [publish_trajectory_node]: ACK: ACCEPTED
[speed_control_node-18] [INFO] [1755489159.073530567] [speed_control_node]: Trajectory point 13: -1.435315 -1.061260 -1.853294 -
1.806778 -1.173636 -0.003997
[speed_control_node-18] [INFO] [1755489159.073536837] [speed_control_node]: Trajectory point 14: -1.422658 -1.010630 -1.826647 -
1.883389 -1.386818 -0.001999
[speed_control_node-18] [INFO] [1755489159.073542887] [speed_control_node]: Trajectory point 15: -1.410000 -0.960000 -1.800000 -
1.960000 -1.600000 0.000000
[gazebo-10] [INFO] [1755489159.073772677] [scaled_joint_trajectory_controller]: Received new action goal
[gazebo-10] [INFO] [1755489159.073805787] [scaled_joint_trajectory_controller]: Accepted new action goal
[speed_control_node-18] [INFO] [1755489159.073993117] [speed_control_node]: Goal accepted by server
[publish_trajectory_node-17] [INFO] [1755489159.074136887] [publish_trajectory_node]: ACK: ACCEPTED
[proximity_sensor_node-19] [INFO] [1755489160.252400347] [proximity_sensor_node]: Dist-size=538.5 mm -> Speed=1.00
[proximity_sensor_node-19] [INFO] [1755489162.352225877] [proximity_sensor_node]: Dist-size=465.4 mm -> Speed=1.00
[proximity_sensor_node-19] [INFO] [1755489164.452222017] [proximity_sensor_node]: Dist-size=392.2 mm -> Speed=1.00
[proximity_sensor_node-19] [INFO] [1755489166.551851087] [proximity_sensor_node]: Dist-size=308.8 mm -> Speed=1.00
[proximity_sensor_node-19] [INFO] [1755489168.551945887] [proximity_sensor_node]: Dist-size=228.6 mm -> Speed=1.00

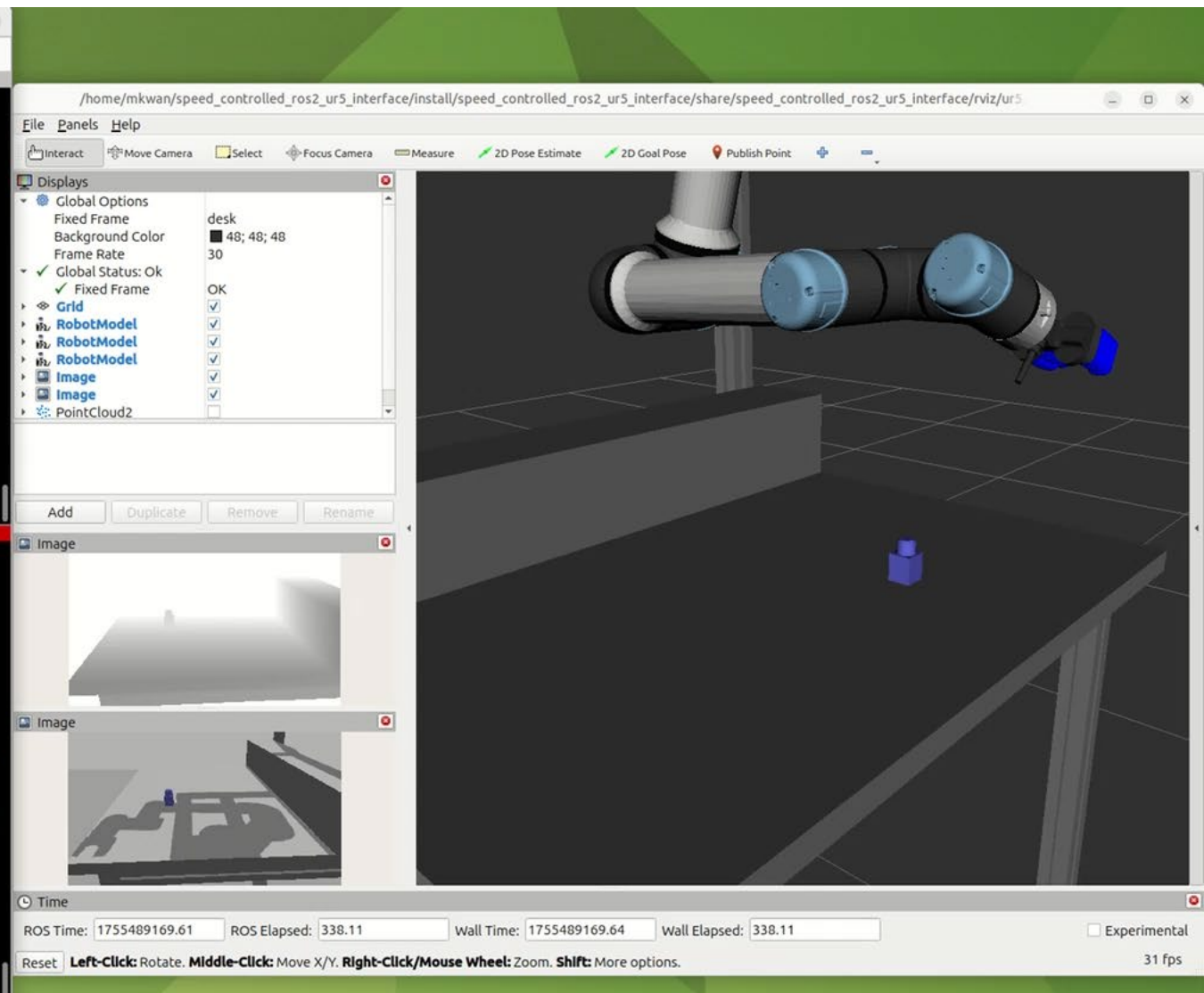
ubuntu@e6fb7a587007: /home/mkwan/speed_controlled_ros2_ur5_interface 115x23
ubuntu@e6fb7a587007: /home/mkwan/speed_controlled_ros2_ur5_interface$ ros2 service call /estop_off std_srvs/srv/Trig
ger {}
waiting for service to become available...
requester: making request: std_srvs.srv.Trigger_Request()

response:
std_srvs.srv.Trigger_Response(success=True, message='E-STOP cleared')

ubuntu@e6fb7a587007: /home/mkwan/speed_controlled_ros2_ur5_interface$ ros2 service call /world/default/set_pose ros_
gz_interfaces/srv/SetEntityPose "{ entity: { name: 'block1', type: 2 }, pose: { position: { x: 0.325, y: 0.58, z: 0
.90 }, orientation: { x: 0.0, y: 0.0, z: 0.0, w: 1.0 } } }"
2025-08-18 03:52:37.803 [RTSP_TRANSPORT_SHM Error] Failed init_port fastrtps_port7002: open_and_lock_file failed ->
Function open_port_internal
requester: making request: ros_gz_interfaces.srv.SetEntityPose_Request(entity=ros_gz_interfaces.msg.Entity(id=0, na
me='block1', type=2), pose=geometry_msgs.msg.Pose(position=geometry_msgs.msg.Point(x=0.325, y=0.58, z=0.9), orienta
tion=geometry_msgs.msg.Quaternion(x=0.0, y=0.0, z=0.0, w=1.0)))

response:
ros_gz_interfaces.srv.SetEntityPose_Response(success=True)

ubuntu@e6fb7a587007: /home/mkwan/speed_controlled_ros2_ur5_interface$
```



Proximity sensor and speed control demonstration. The cobot slows then stops when approaching the block. When the block is moved, the cobot no longer detects its proximity and resumes full-speed operation. Shown twice.

```
ubuntu@e6fb7a587007:/home/mkwan/speed_controlled_ros2_ur5_interface
ubuntu@e6fb7a587007:/home/mkwan/speed_controlled_ros2_ur5_interface x ubuntu@e6fb7a587007:/opt/ros/jazzy/share/controller_manager_msgs/srv
ubuntu@e6fb7a587007:/home/mkwan/speed_controlled_ros2_ur5_interface 128x24
[speed_control_node-18] [INFO] [1755488891.527906923] [speed_control_node]: Trajectory point 8: -1.498596 -1.314382 -1.986513 -1.423766 -0.107843 -0.013988
[speed_control_node-18] [INFO] [1755488891.527919133] [speed_control_node]: Trajectory point 9: -1.485940 -1.263756 -1.959868 -1.500371 -0.321008 -0.011990
[speed_control_node-18] [INFO] [1755488891.527929363] [speed_control_node]: Trajectory point 10: -1.473283 -1.213130 -1.933223 -1.576976 -0.534174 -0.009992
[speed_control_node-18] [INFO] [1755488891.527938713] [speed_control_node]: Trajectory point 11: -1.460626 -1.162504 -1.906579 -1.653581 -0.747339 -0.007993
[speed_control_node-18] [INFO] [1755488891.527948173] [speed_control_node]: Trajectory point 12: -1.447970 -1.111878 -1.879934 -1.730186 -0.960504 -0.005995
[speed_control_node-18] [INFO] [1755488891.527958293] [speed_control_node]: Trajectory point 13: -1.435313 -1.061252 -1.853289 -1.806790 -1.173669 -0.003997
[speed_control_node-18] [INFO] [1755488891.527967973] [speed_control_node]: Trajectory point 14: -1.422657 -1.010626 -1.826645 -1.883395 -1.386835 -0.001998
[speed_control_node-18] [INFO] [1755488891.527985793] [speed_control_node]: Trajectory point 15: -1.410000 -0.960000 -1.800000 -1.960000 -1.600000 0.000000
[publish_trajectory_node-17] [INFO] [1755488891.527766893] [publish_trajectory_node]: ACK: ACCEPTED
[gazebo-10] [INFO] [1755488891.528225373] [scaled_joint_trajectory_controller]: Received new action goal
[gazebo-10] [INFO] [1755488891.528253313] [scaled_joint_trajectory_controller]: Accepted new action goal
[speed_control_node-18] [INFO] [1755488891.528395323] [speed_control_node]: Goal accepted by server
[publish_trajectory_node-17] [INFO] [1755488891.528494763] [publish_trajectory_node]: ACK: ACCEPTED
[proximity_sensor_node-19] [INFO] [1755488893.351954533] [proximity_sensor_node]: Dist-size=660.1 mm -> Speed=1.00
[proximity_sensor_node-19] [INFO] [1755488895.451978833] [proximity_sensor_node]: Dist-size=625.3 mm -> Speed=1.00

ubuntu@e6fb7a587007:/home/mkwan/speed_controlled_ros2_ur5_interface 115x23
ros_gz_interfaces.srv.SetEntityPose_Response(success=True)

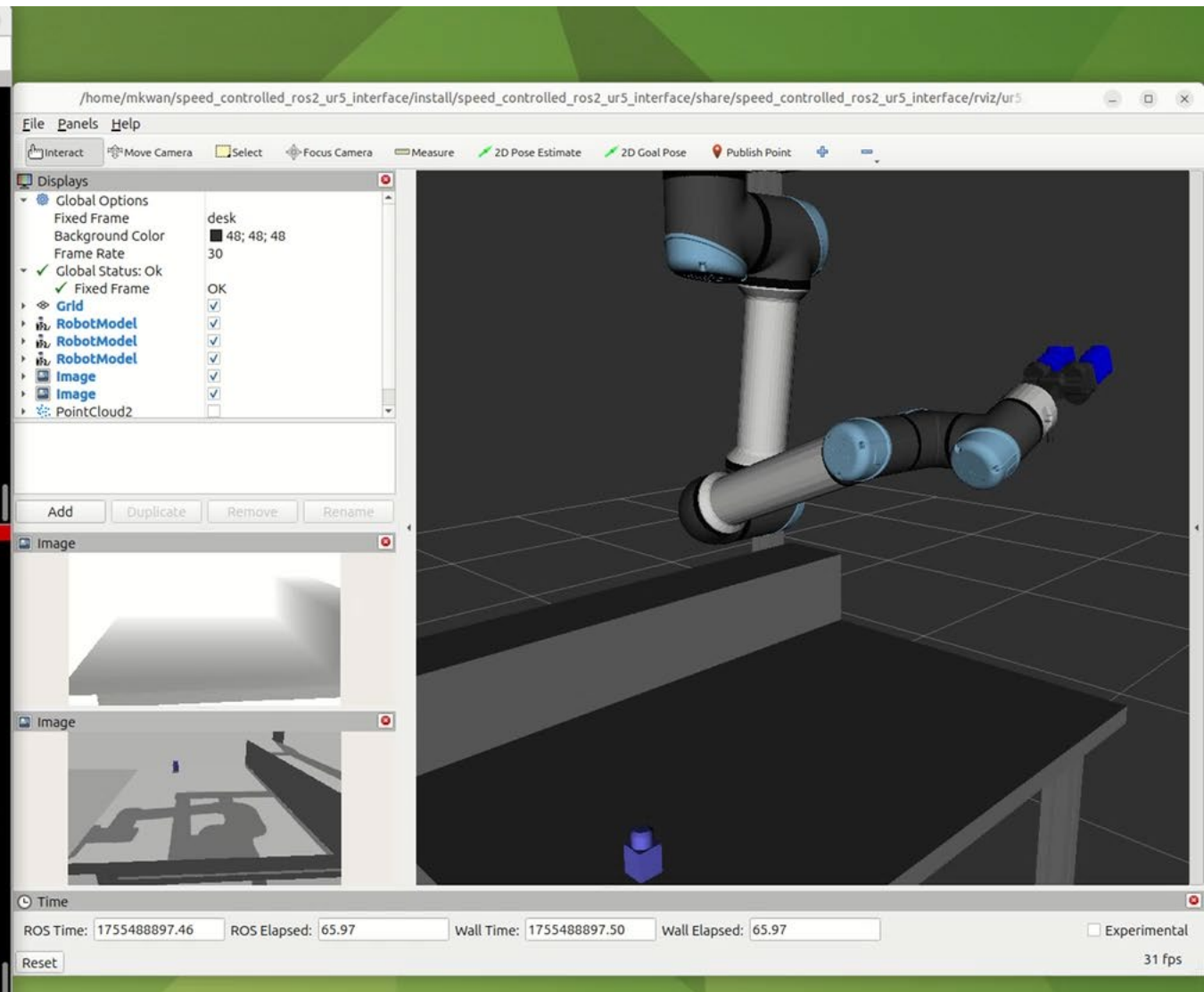
ubuntu@e6fb7a587007:/home/mkwan/speed_controlled_ros2_ur5_interface$ ros2 service call /estop_off std_srvs/srv/Trigger {}
requester: making request: std_srvs.srv.Trigger_Request({})

response:
std_srvs.srv.Trigger_Response(success=True, message='E-STOP cleared')

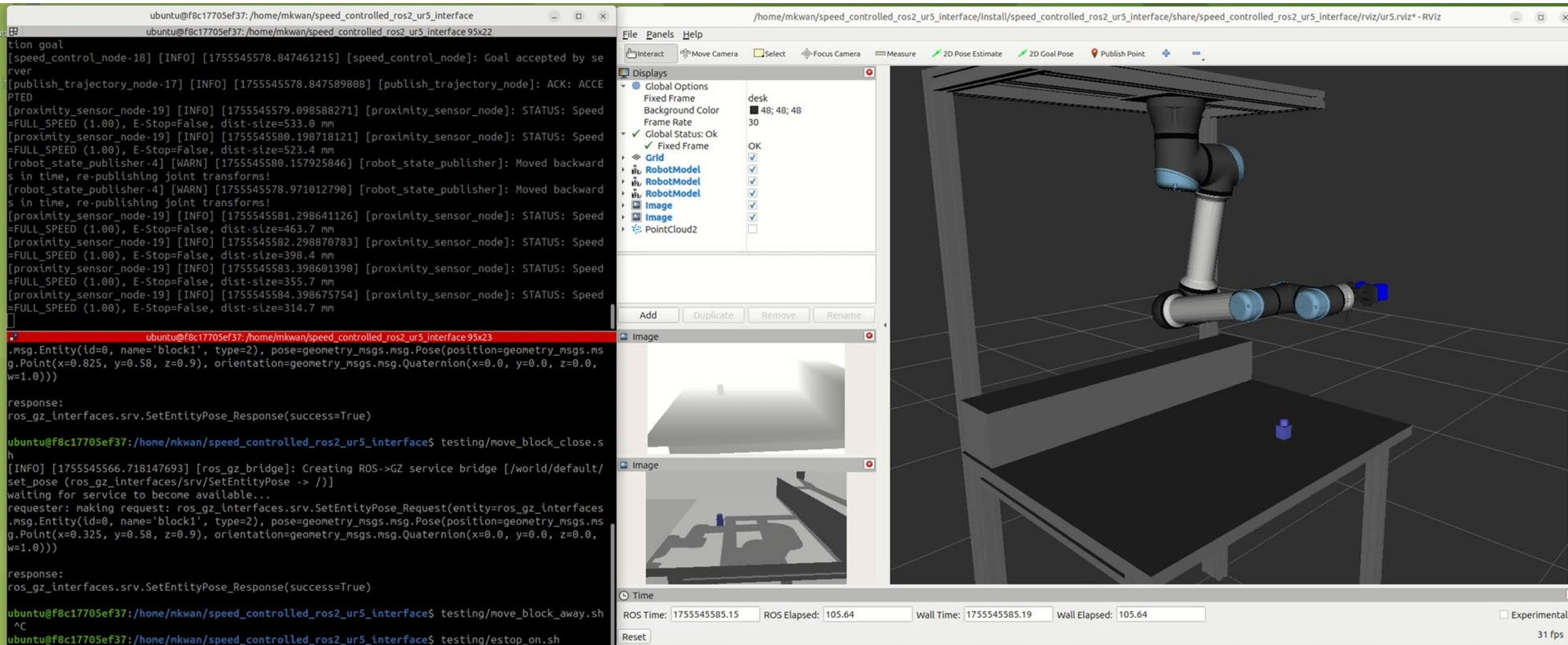
ubuntu@e6fb7a587007:/home/mkwan/speed_controlled_ros2_ur5_interface$ ros2 service call /world/default/set_pose ros_gz_interfaces/srv/SetEntityPose "{ entity: { name: 'block1', type: 2 }, pose: { position: { x: 0.825, y: 0.58, z: 0.90 }, orientation: { x: 0.0, y: 0.0, z: 0.0, w: 1.0 } } }"
waiting for service to become available...
requester: making request: ros_gz_interfaces.srv.SetEntityPose_Request(entity=ros_gz_interfaces.msg.Entity(id=0, name='block1', type=2), pose=geometry_msgs.msg.Pose(position=geometry_msgs.msg.Point(x=0.825, y=0.58, z=0.9), orientation=geometry_msgs.msg.Quaternion(x=0.0, y=0.0, z=0.0, w=1.0)))

response:
ros_gz_interfaces.srv.SetEntityPose_Response(success=True)

ubuntu@e6fb7a587007:/home/mkwan/speed_controlled_ros2_ur5_interface$ ros2 service call /estop_on std_srvs/srv/Trigger {}
```



E-Stop demonstration. A ROS2 service activates the E-Stop and another ROS2 service deactivates the E-Stop. When the E-Stop is activated, the cobot freezes in place. When the E-Stop is cleared, operation resumes normally.



Full scenario demonstration. (1) Proximity sensing triggers a speed change. (2) E-Stop interrupting operation during proximity detection. (3) Once both the obstacle and the E-Stop are cleared, normal operation resumes. Shown twice.

Discussion & Conclusion

- Key decisions:
 - Integrating with UR5 Simulator has significant development cost but is more representative and would likely transfer decently onto real UR5 hardware
 - ROS2's controllers mandate non-blocking concurrency which allow better scaling and elasticity for more complex processing and/or constrained hardware
 - The E-Stop does not simply command a 0.0 speed; it disconnects the entire joint trajectory controller to further ensure motion commands will fail, in addition to setting a 0.0 speed and toggling an E-Stopped state across the platform
 - Using a circular buffer for hysteresis limits noisy proximity data
- Future work:
 - Realistic object detection via LiDAR or stereo vision point cloud
 - Easy incremental change would be to add random noise to the proximity information
 - Modular unit testing with deterministic output for CI/CD
 - Tuning and stress-testing speed limits for cobot via points per trajectory, base time per point, joint and time tolerances, communication limits
 - Improve exception handling, error state handling, edge cases, complex and/or highly specific trajectories

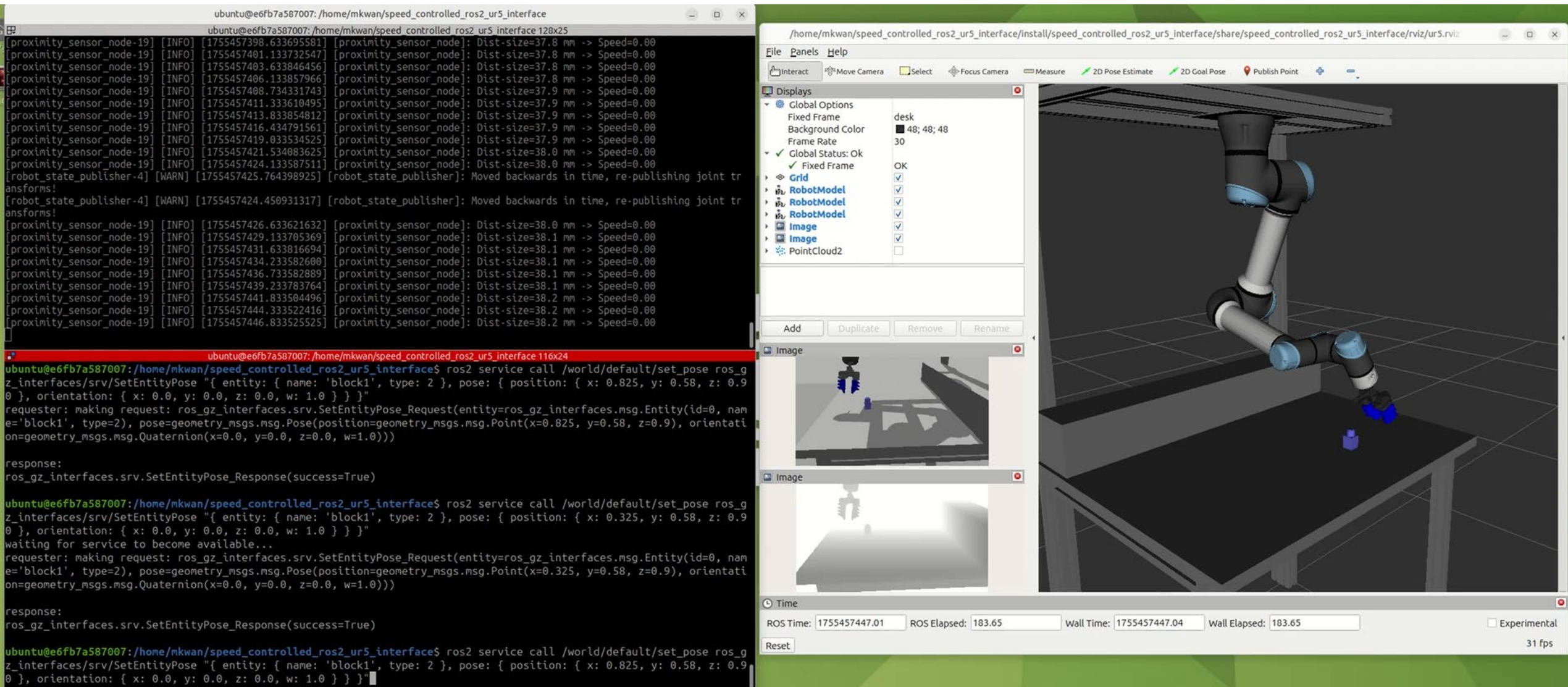
Thank you!

Presented by: Matt Kwan

[LinkedIn](#)

Link to [Github repo](#)

Appendix



Similar demonstration of the proximity sensor integration. Zoomed-out perspective.